

Otto is the orchestration layer

Not one agent sitting in the lifecycle — the platform every agent plugs into. One path takes any request in, routes it, gates the risky steps for a human, runs it, and records what happened.

 Otto core — the path every request takes  AI capability — a routed agent or skill  Human gate — approval on writes

HOW WORK ARRIVES — THREE ADAPTERS, ONE WORKFLOW

Web chat

interactive

A person is watching, so Otto can clarify before it acts.

Schedules

unattended · time

Cron via Temporal. No audience — skips clarify, writes only if the trigger pre-approves.

Events & webhooks

unattended · reactive

Signed inbound events map to a request, then deliver the result to a target.



Otto — one path for every request

01
Route
Pick the most specific capability for the request.

03
Risk gate
Reads run on their own. Writes stop here.

👉 human approval

05
Verify → retry
Judge the result, fold critique back, escalate the model on the last try.

02
Clarify
Ask a follow-up — only when someone's there to answer.

04
Run
`claude -p` drives the real tools and subagents.

06
Remember & record
Distil durable facts to memory; append the run to the audit log.

Model gateway maps each phase — route · clarify · memory · verify · execution — to the cheapest capable model. Execution stays Claude.

Adding a capability is a registry entry, not a new system.



CAPABILITIES PLUG IN AS TILES

Agents that would otherwise sit side by side across the lifecycle become routed capabilities behind that single path. The lifecycle stages don't go away — they're how the tiles are grouped.

Research

Idea → validated direction

deep-research

tech-investigation

empirical-investigation

Feature gathering

Requests → triaged work

github-issue

product-manager

github-pr-review

Planning

Work → implementation plan

design-doc

Plan

Implementation

Issue → PR → self-review

implementer

code-review

Build · test · deploy

CI/CD pipeline

ci-deploy

ci-build-status

deploy-status

Ops & monitoring

Cloud & cluster health

incident

k8s-health

aws-health

incident-inspector

Metrics & reporting

How the team & platform perform

daily-summary

board-status

work-checkin



EXTERNAL SYSTEMS what Otto *calls* — not part of the platform. Capabilities reach these through `claude -p` tools and MCP servers.

CI/CD

AWS

Kubernetes / EKS

GitHub

New Relic

Grafana

Slack

PagerDuty

ABOUT THE TILES

The capability tiles are an *example* landscape — one team's agents and skills, not what Otto ships. Otto discovers whatever lives in your own `~/claude`; only the handful under `capabilities/` come with it.

The point of the drawing is that the agents aren't peers of Otto — they run *inside* it. Otto is the connective tissue: ingress, routing, the write-approval gate, the verify loop, and the memory + audit trail are shared by every capability. So the lifecycle is open-ended: a new agent is a tile dropped into the registry, and it inherits routing, gating, verification and audit for free.